

Sunum Konuşma Scripti

Akıllı Otopark Yönlendirme Sistemi · LLM + IoT

Hazırlayan: Mehdi Sındağ · Mehmet Ali Topkara — Nesnelerin Yapay Zekâsı (AloT)

Nasıl kullanılır: Her slayt için ~30–40 saniyelik akıcı bir anlatım var. Ezberlemek yerine anla-anlat: her bölümün altındaki **Terimler** notu, jargonu (MQTT, A*, LLM, maliyet fonksiyonu) açabilmen için. Sonda Terimler Sözlüğü ve Olası Sorular bölümü var. İki kişiyseñiz slaytları paylaşın (ör. 1–5 / 6–10); demoyu birlikte yapın.

Slayt 1 — Kapak / Giriş (~30 sn)

Merhaba, biz Mehdi Sındağ ve Mehmet Ali Topkara. Yapay Zekâ ve Veri Mühendisliği bölümünden, **Nesnelerin Yapay Zekâsı** dersi için **Akıllı Otopark Yönlendirme Sistemi**'ni hazırladık. AloT, klasik nesnelerin internetinin üstüne yapay zekâ ekleyen yaklaşımdır. Sistemin özeti tek cümlede şu: **sürücü doğal dille konuşur, bir dil modeli bunu anlar, algoritma en uygun park yerini seçer ve sonuç web arayüzünde canlı gösterilir**. Şimdi böyle bir sisteme neden ihtiyaç olduğuna bakalım.

Terimler: AloT = Artificial Intelligence of Things (Nesnelerin Yapay Zekâsı): IoT cihazlarının topladığı veriye yapay zekâ ile karar/anlam katmak.

Slayt 2 — Problem & Motivasyon (~35 sn)

Problemimiz herkesin yaşadığı bir şey: büyük AVM otoparklarında **boş yer aramak**. Bu hem vakit hem yakıt kaybı, hem de içeride gereksiz trafik yaratır. İkinci nokta: sürücü teknik bir form doldurmak yerine **insan gibi konuşsun** — “elektrikli arabam var, 2 saat kalacağım” desin. Üçüncüsü: **IoT** sayesinde sensörler doluluğu canlı biliyor; ama bu ham veriyi anlamlı bir karara dönüştürmek gerekiyor. Hedefimiz bu üçünü — doğal dil, IoT verisi ve algoritmayı — tek bir çalışan sistemde birleştirmek. Donanım yok; sensörleri simüle ediyoruz ama **mimari tamamen gerçekçi**.

Terimler: IoT = Internet of Things (Nesnelerin İnterneti): fiziksel cihazların (sensörler) internet üzerinden veri paylaşması.

Slayt 3 — Çözüm: Ne Yaptık (~40 sn)

Çözümümüz **konuşan asistanlı** bir otopark. Uçtan uca akış şöyle: sürücü doğal dille yazar veya konuşur; **LLM** isteği anlar ve **function calling** ile — yani metni yapılandırılmış bir fonksiyon çağrısına çevirerek — bizim algoritmamızı tetikler; algoritma **A*** ve maliyet fonksiyonuyla en uygun boş yeri seçer; sonuç hem doğal dille açıklanır hem haritada canlı gösterilir. Sistem **üç katman**: IoT (sensör simülasyonu, MQTT, SQLite), Algoritma (graf, A*, maliyet fonksiyonu, Hungarian) ve LLM (doğal dili yapılandırılmış çağrıya çeviren kısım).

Terimler: LLM = Large Language Model (Büyük Dil Modeli, ör. Gemini). Function calling = modelin serbest metni, parametreleri belli bir fonksiyon çağrısına dönüştürmesi.

Slayt 4 — Mimari (~40 sn)

Mimarimiz **gevşek bağlı ve katmanlı**. Solda sensör simülatörü — üstünde Edge AI ve sağlık telemetrisi — veriyi **MQTT** protokolüyle yayınlıyor. MQTT, IoT'nin standart, hafif mesajlaşma protokolüdür; açılımı Message Queuing Telemetry Transport. Topic hiyerarşimiz var: otopark/spots, otopark/health, otopark/gateway. Ortada backend: SQLite, A* + maliyet, LLM, anomali ve tahmin. Arayüzle **REST ve WebSocket** üzerinden konuşuyor. Her katman bağımsız, servis tabanlı. Broker çalışmasa bile sistem **brokersız yedeğe** düşüp çalışmaya devam ediyor — buna graceful degradation (zarif bozulma) diyoruz.

Terimler: MQTT = hafif publish/subscribe (yayınla/abone ol) protokolü. Broker = mesajları dağıtan aracı sunucu. REST/WebSocket = arayüzün veri çekme (REST) ve canlı akış (WS) yolları.

Slayt 5 — IoT Katmanı (~40 sn)

IoT katmanında gerçek bir sensör topolojisi kurduk. MQTT'yi profesyonelce kullanıyoruz: **hiyerarşik topic'ler**, **QoS 1** — mesajın en az bir kez ulaşması garantisi — ve **retained** (son durumu saklayan) mesaj. **Last Will (LWT)**: ağ geçidi çökerse broker otomatik “offline” duyurusu yapıyor. Her yerin pil, sinyal, çevrimiçi telemetrisi var; sistem çevrimdışı, takılı veya düşük pilli sensörleri **anomali** olarak yakalıyor — bu kestirimci bakım demek. Veriyi **SQLite**'ta tutuyoruz, gelen her mesajı **Pydantic** ile doğruluyoruz; bozuk veri reddediliyor. Ayrıca gün-içi doluluk simülasyonu: gece boş, öğle/akşam zirve; bir simüle gün yaklaşık 4 dakika.

Terimler: QoS = Quality of Service (teslim garantisi seviyesi). LWT = Last Will & Testament (cihaz beklenmedik koparsa broker'ın yayınladığı vasiyet mesajı). SQLite = tek dosyalık, sunucusuz veritabanı. Pydantic = Python veri doğrulama kütüphanesi.

Slayt 6 — Edge AI (Uç Zekâ) (~40 sn) [dersin kalbi]

Bu slayt dersin kalbi: **Edge AI**, yani uç zekâ. Klasik IoT'de ham veri merkeze gider, kararı merkez verir. Edge AI'da ise **basit kararı cihazın kendisi** verir. Sorunumuz **gürültü**: park sensörü, üstünden geçen bir yayaya veya yavaş geçen araca da tepki verir. Bu kısa sıçramalar gerçek park değil; merkeze gönderilirse sistem titrer. Çözümümüz uçta **debounce**: sensör hemen haber vermez, kısa süre bekler; sinyal birkaç tur kararlı kalırsa “gerçek park” deyip yayınlar. Yani **park kararını bulutta LLM verir; bu sinyal gerçek mi gürültü mü kararını sensör uçta verir** — zekâ hem merkezde hem nesnenin kendisinde.

Terimler: Edge AI = veriyi merkeze yollamadan, cihazın (uç/edge) kendi üstünde karar vermesi. Debounce = kısa, kararsız sinyalleri stabil kalana kadar bekleyerek eleme tekniği.

Slayt 7 — Algoritma Katmanı (~40 sn)

Kararı veren **deterministik çekirdek** burası. Otoparkı bir **graf** olarak modelledik: girişler, çıkışlar, koridorlar ve park yerleri düğümler. **A*** ile her aday yere en kısa sürüş mesafesini buluyoruz; araç tipi, şarj ve engelli filtresini uyguluyoruz. Aynı anda birden çok araç gelirse **Hungarian (Macar algoritması)** ile çakışmadan, toplam maliyeti en aza indirecek şekilde dağıtıyoruz; iki araç asla aynı yere gönderilmiyor. Kalbinde **maliyet fonksiyonu** var: $C = |d - \alpha \cdot t|$. Burada d girişten mesafe, t kalış süresi, α bir ağırlık katsayısı. Yani **kısa kalan kapıya yakın** (yüksek sirkülasyon), **uzun kalan daha derine** gider; değerli kapı önleri boş kalmaz.

Terimler: A* (“A-star”) = başlangıçtan hedefe en kısa yolu sezgisel tahminle hızlı bulan yol-bulma algoritması. Hungarian = N işi N kaynağa en düşük toplam maliyetle eşleştiren optimal atama algoritması. — Formül $C = |d - \alpha \cdot t|$: araç t saat kalacaksa ideal mesafe $\alpha \cdot t$ olsun isteriz; bir yerin maliyeti gerçek mesafe d ile bu ideal arasındaki farktır; en küçük farklı yer seçilir.

Slayt 8 — LLM Katmanı (~40 sn)

LLM katmanında ilkemiz net: **LLM karar VERMEZ**. Doğal dili anlar, doğru aracı çağırır ve sonucu açıklar; park kararını deterministik algoritma verir. **Function calling** ile üç aracımız var: `find_best_parking_spot` yer bulur, `get_parking_stats` “kaç boş yer var” sorusunu yanıtlar, `predict_availability` “birazdan dolar mı” tahminini yapar. Türkçe ve İngilizce çalışıyor; **Gemini** modelini kullanıyoruz. Kota veya erişim sorunu olursa **anahtar-kelime yedeğine** düşüyor, sistem yine çalışıyor. Örnek: sürücü “elektrikli arabam var, 2 saat kalacağım” der; LLM bunu `vehicle_type=ev, duration=2`'ye çevirir; algoritma B-12'yi seçer; LLM de “sizi B-12 şarjlı yere yönlendirdim” diye açıklar.

Terimler: Neden LLM karar vermiyor? Dil modelleri uydurabilir (halüsinasyon). Kararı deterministik algoritmaya bırakınca sonuç her zaman doğru ve açıklanabilir; LLM'i sadece anlama ve açıklama için kullanıyoruz. Sunumun en güçlü mesajı budur.

Slayt 9 — Ek Özellikler & Mühendislik (~40 sn)

Çekirdeğin üstüne **mühendislik olgunluğu** kattık. **Analitik panel:** doluluk zaman serisi, bölge oranları, ısı haritası, ortalama kalış süresi. **Tahmin:** eğilim ve gün-içi örüntüyle “15 dakika sonra yüzde kaç dolu”. **Rezervasyon:** yeri gelmeden ayırtabiliyorsunuz, zaman aşımında otomatik temizleniyor. **Sesli giriş:** Web Speech API ile konuşarak istek. Veri doğrulama için **Pydantic**, her yer için seviyeli **loglama**. **Hata toleransı:** MQTT yeniden bağlanma, LLM yedeği, brokersız mod, thread kilidi. Tüm bunları onlarca birim testi ve uçtan uca canlı doğrulamayla destekledik.

Terimler: Isı haritası = her park yerinin ne sıklıkta dolu olduğunu renkle gösteren görsel. Logging = sistemin ne yaptığını seviyeli (bilgi/uyarı/hata) kaydetmesi. (Slaytta 37 test yazıyor; kodun güncel halinde 43 var — istersen “kırktan fazla” diyebilirsin.)

Slayt 10 — Sonuç & Demo (~35 sn)

Özetle: **doğal dil, IoT ve algoritmayı** tek bir çalışan sistemde birleştirdik. **Üç tür zekâyı** bir arada gösterdik: uçta sensör debounce, algoritmik tarafta A* artı maliyet fonksiyonu, dil tarafında LLM. Çekirdek mesajımız: **LLM anlar ve açıklar, ama kararı deterministik algoritma verir** — yapay zekâyı süs değil, fonksiyonel bir bileşen olarak kullandık. Şimdi demoya geçiyoruz: normal, engelli ve elektrikli araç senaryoları, aynı anda çoklu araç ataması, doluluk tahmini ve analitik paneli canlı göstereceğiz. **Teşekkürler — sorularınızı almaktan memnuniyet duyuyoruz.**

Terimler Sözlüğü (hızlı bakış)

IoT	Internet of Things — Nesnelerin İnterneti; cihazların veri paylaşması
AIoT	Artificial Intelligence of Things — IoT verisine yapay zekâ ile karar katmak
LLM	Large Language Model — Büyük Dil Modeli (Gemini)
Function calling	LLM'in metni, parametrelili bir fonksiyon çağrısına çevirmesi
MQTT	Message Queuing Telemetry Transport — hafif publish/subscribe IoT protokolü
Broker	MQTT mesajlarını yayıncıdan abonelere dağıtan aracı sunucu
QoS 1	Quality of Service düzey 1 — mesajın en az bir kez teslimi garantisi
Retained	Broker'ın bir topic'in son mesajını saklayıp yeni aboneye anında vermesi
LWT	Last Will & Testament — cihaz koparsa yayınlanan offline mesajı
Edge AI	Uç zekâ — kararın merkez yerine cihazın kendisinde verilmesi
Debounce	Kararsız/kısa sinyalleri stabil olana dek bekleyerek eleme
A*	“A-star” — sezgisel destekli en kısa yol bulma algoritması
Hungarian	Macar/Kuhn-Munkres — en düşük toplam maliyetli optimal eşleştirme
Maliyet fonk.	$C = d - \alpha \cdot t $ — yeri kalış süresine göre seçen formül
SQLite	Tek dosyalık, sunucusuz ilişkisel veritabanı
Pydantic	Python veri doğrulama kütüphanesi (bozuk veriyi reddeder)
REST / WS	Arayüzün veri çekme (REST) ve canlı akış (WebSocket) yöntemleri
Graceful degradation	Bir parça çökse de sistemin çalışmaya devam etmesi

Olası Sorular (hazırlık)

- **Neden A*, basit en-yakın değil?** Otopark bir yol ağı; düz çizgi değil gerçek sürüş mesafesi gerekiyor. A* sezgisel sayesinde en kısa rotayı hızlı bulur.
- **Neden MQTT, doğrudan veritabanı değil?** MQTT, gerçek IoT'nin sensör → broker → tüketici mimarisini gösterir; gevşek bağlı, ölçeklenebilir ve canlı (push) çalışır.
- **LLM yanlış anlarsa?** Karar zaten LLM'de değil; ayrıca kota/erişim sorununda anahtar-kelime yedeği devreye girer, sistem çalışmaya devam eder.
- **α (alfa) nasıl seçildi?** Otoparkın fiziksel büyüklüğüne göre; en derin mesafe en uzun kalış süresine denk gelecek şekilde ayarlandı (koddaki ALPHA_DISTANCE_PER_HOUR).
- **Gerçek sensör yokken IoT sayılır mı?** Sensör simüle; ama protokol (MQTT/QoS/LWT), topoloji, edge debounce ve anomali tamamen gerçekçi — kolayca gerçek donanıma takılır.
- **Edge AI tam olarak nerede?** Sensör düğümünde: ham doluluğu merkeze yollamadan önce debounce ile gürültüyü uçta eler; sadece gerçek park olaylarını yayınlar.